CAREER · PART 30

"Design ChatGPT for Our Company Docs."

The AI system design interview. The 45-minute format, a framework you can repeat under pressure, the math interviewers expect, and the pushes they use to separate seniors from juniors.

`Requirements` `Numbers` `Architecture` `Trade-offs` `Evals`

16 slides · framework · worked math · reference design · deep dives · question bank



AI Coach John Follow

PROITBRIDGE
Strive For Better Future

Repost

What the 45 minutes actually test

It is not a RAG quiz. The interviewer wants to watch you turn a vague product ask into a system with numbers, boundaries and a plan to know if it works.

5 min	5 min	10 min	15 min	5 min	5 min
Clarify	Numbers	High-level design	Deep dives	Evals	Close

TYPICAL PROMPT

```
"Employees ask questions about 1M internal docs. Design it. Budget matters."
```

WHAT GETS SCORED

Problem framing
Did you ask about users, data, latency and permissions before drawing a single box?

Quantitative sense
Can you estimate chunks, vectors, QPS and monthly cost within an order of magnitude?

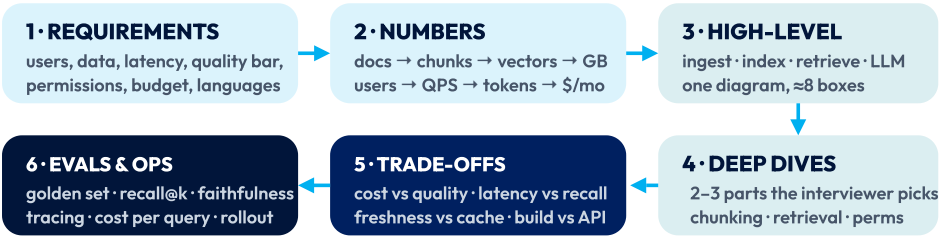
Trade-off fluency
Every choice named with what it costs. "Rerank improves precision, adds ≈150 ms."

Ownership
Evals, monitoring, rollout and failure handling. Seniors bring these up unprompted.

FRAMEWORK

One framework for every AI design question

Six steps, always in this order. Say the step names out loud. Interviewers relax when they can see the structure.



TIME SPLIT

5 · 5 · 10 · 15 · 5 · 5 minutes. Deep dives get the most time, because that is where seniority shows.

Narrate the plan

"I will clarify, size it, draw the pipeline, then go deep where you want. Sound good?" Ten seconds, huge signal.

Loop back

After each deep dive, return to the diagram and update it. Shows the design is one system, not a list.

Clarifying questions that impress

Juniors ask "which vector DB?" Seniors ask questions whose answers change the architecture.

Q1 **Who asks and how often?** 500 employees or 5M customers changes everything: QPS, cost model, abuse surface.

Q2 **How much data and what shape?** 1M PDFs vs 50K wiki pages vs a SQL warehouse. Parsing is half the project.

Q3 **How fresh must answers be?** Nightly re-index or seconds? Decides streaming ingest vs batch and whether caching is safe.

Q4 **Latency target?** p95 under 3 s for chat, 30 s fine for a report. Sets your rerank and model-size budget.

Q5 **Who may see what?** If HR docs exist, permissions are enforced at retrieval, not in the prompt. Say this early.

Q6 **Budget and data residency?** Hosted API vs self-hosted, region pinning, whether data can leave the VPC.

Q7 **What does "good" look like?** Citations required? Refuse when unsure? Multilingual? This becomes your eval set.

Write answers as constraints: "5K users · 1M docs · nightly · p95 3 s · role-based access · $5K/month". Point at this list for 40 minutes.

Back-of-envelope math, worked

Interviewers do not check your arithmetic. They check that you know which quantities matter and roughly how big they get.

STORAGE · 1M DOCS

1M docs × 5 pages → **2.5B tokens** ≈500 tok/page → **5M chunks** 500 tok each → **≈15 GB** 768-d fp32 vectors → **≈30 GB** +HNSW +text

TRAFFIC · 5,000 EMPLOYEES

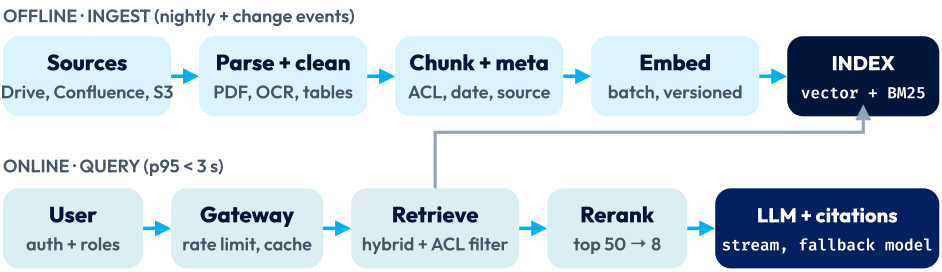
25K	≈0.3	≈5K	125M
queries/day at 5 per user	average QPS, peak ≈3	tokens per query (4K in, 500 out)	tokens per day

COST LINE	ESTIMATE	HOW YOU GOT IT
One-time embedding	≈$50–250	2.5B tokens at $0.02–0.10 per 1M
LLM per month, mid-tier API	≈$1–4K	125M tok/day × 30 × blended $0.3–1 per 1M
Self-host alternative	≈$2–3K	1 GPU node 24/7, only wins at higher volume
Vector store	≈$100–500	30 GB managed, or pgvector on existing Postgres

Say the conclusion: "≈3 QPS peak and 30 GB. One Postgres + pgvector and a hosted API. No GPU cluster needed." That sentence is what they wanted.

The reference architecture

Draw this in under three minutes. Two paths: an offline ingest pipeline and an online query path, joined by the index.



CROSS-CUTTING tracing every hop · semantic cache · golden-set evals · feedback → weekly review · cost/query

SAY OUT LOUD "Permissions are a filter inside retrieval, so a chunk you cannot read never reaches the prompt."

Eight boxes, two arrows into the index, one dashed cross-cutting bar. Anything more on the first pass and you run out of time for deep dives.

Deep dive 1: ingestion

"How do you get a million PDFs into that index, and keep it current?" Answer with a pipeline, a chunking choice with a reason, and an update strategy.

STAGE	STRONG ANSWER	NUMBER TO SAY
Parsing	Layout-aware PDF parser, OCR fallback for scans, tables to markdown	`≈5–10% docs need OCR`
Chunking	Semantic or heading-based, 300–800 tokens, 10–15% overlap, parent doc kept	`≈5 chunks per doc`
Metadata	source, url, updated_at, owner, ACL groups, doc type, language	`ACL is mandatory`
Embedding	Batch API, versioned model name stored per chunk, retry queue	`1000s of chunks/s`
Updates	Change events + content hash, re-embed only changed chunks, tombstones for deletes	`nightly + near-real-time`

Full rebuild
simple, days for 1M docs
only when model changes

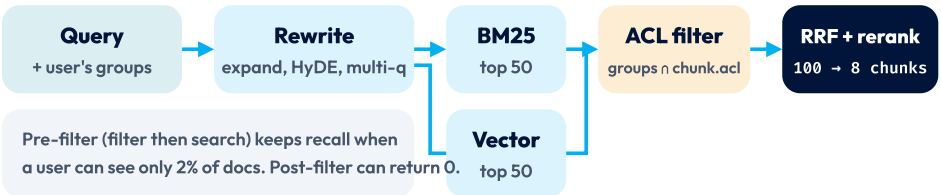
Incremental by hash
re-embed ≈1–2% per night
the default answer

Event-driven
`webhook → queue → embed`
for "seconds" freshness

Mention the embedding-model version trap: changing models means re-embedding everything, so store the model name per chunk and run blue/green indexes.

Deep dive 2: retrieval and permissions

"Why not just vector search?" and "How do you stop an intern seeing the CFO's docs?" These two come up in almost every loop.

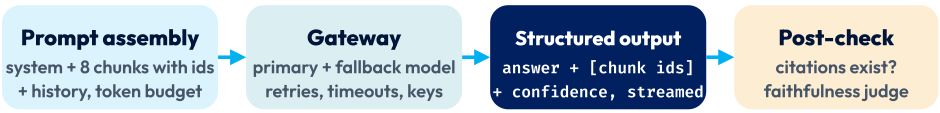


Cross-encoder rerank adds ≈100–300 ms and clearly lifts precision@5. Worth it inside a 3 s budget.

PUSH	ANSWER THAT LANDS
Why hybrid?	Vectors miss exact IDs, SKUs and names. BM25 misses paraphrase. Fuse with reciprocal rank fusion.
Permissions?	ACL groups stored on each chunk at ingest, applied as a filter inside the index query. Never trust the LLM to redact.
Multi-tenant?	tenant_id as a partition key or namespace. Separate indexes for large or regulated tenants.
ACL changed?	Sync group membership hourly, doc ACL changes flow through the same update pipeline as content.

Deep dive 3: generation and hallucination

"How do you stop it making things up?" Layer the answer: better context in, constrained output, a check after, and an honest refusal path.



1. **Grounding rule in the system prompt.** "Answer only from the provided passages. If they do not contain the answer, say so." Cheap, cuts a large share of fabrications.

2. **Citations by chunk id, verified in code.** Every claim carries [3]. If the id is not in the retrieved set, the answer is rejected or regenerated.

3. **Retrieval confidence gate.** Top rerank score below threshold → "I could not find this in our docs" plus a search link. No answer beats a wrong one.

4. **Sampled LLM-as-judge for faithfulness.** Score ≈5% of production answers offline. Trend it. Alert on drops.

Latency budget, p95 3 s

retrieve 150 ms · rerank 200 ms · TTFT 600 ms · stream 500 tokens ≈ 2 s. Streaming is what makes it feel fast.

Fallbacks

Primary model down → smaller model, same prompt. Retrieval down → answer refused, never "from memory".

10 /16 PRESSURE

Seven pushes interviewers always use

They are testing whether you have operated one of these. Each answer should name a mechanism and a number.

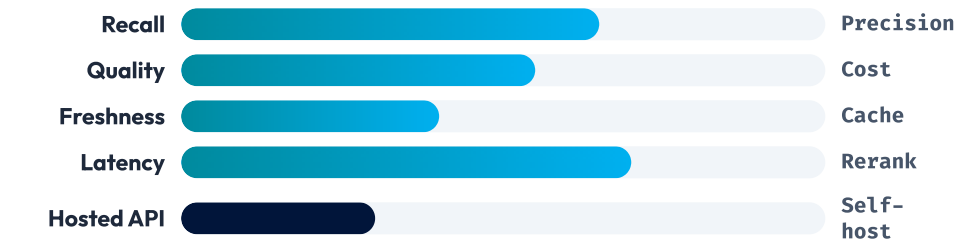
THE PUSH	THE ANSWER THAT LANDS
"Answers are slow."	Profile per hop. Cache embeddings and frequent queries, rerank fewer candidates, stream, use a smaller model for easy queries.
"Now 10× the docs."	Shard by tenant or source, HNSW with quantization (int8 cuts 4×), move to a dedicated vector DB, parallel ingest workers.
"Now 100× the users."	Gateway rate limits, semantic cache (≈20–40% hit rate on FAQ traffic), async queue for reports, cost caps per team.
"Docs change hourly."	Event-driven ingest, short cache TTL keyed on doc version, updated_at boost in ranking.
"It leaked a salary doc."	ACL filter is inside the retriever query. Audit log of chunk ids per answer. Red-team prompt set in CI.
"Cost doubled overnight."	Per-request token logging, alert on p95 prompt size, cap history, prompt caching, route easy queries to a cheap model.
"New model came out."	Offline eval on the golden set, shadow traffic, canary 5%, compare faithfulness and cost, then promote. Pin versions.

Pattern: restate the push, name the bottleneck, give the mechanism, give the number, name the trade-off. Twenty seconds each.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

The trade-off vocabulary

Every design decision is a slider. Naming both ends and where you set it, and why, is the single biggest seniority signal in the room.



Recall vs precision

Retrieve wide (top 50 hybrid) for recall, rerank narrow (top 8) for precision. Say "two-stage" and they nod.

Quality vs cost

Route: classifier picks a small model for lookups, a large model for synthesis. ≈60–70% of traffic is cheap.

Freshness vs cache

Cache key includes index version. Any re-index invalidates. TTL matches your freshness requirement, not a default.

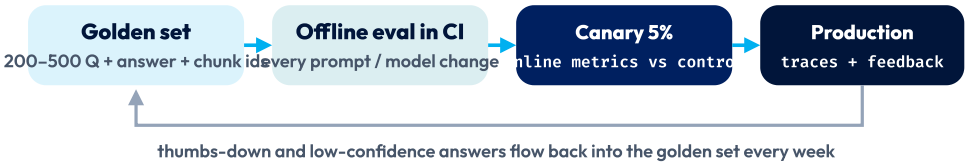
API vs self-host

Under ≈50M tokens/day the API is cheaper once you count engineers. Self-host for residency or volume.

> **Template sentence: "I would set X here because the requirement was Y. If Y changed to Z, I would move it to W." Use it five times.**

How to talk about evals and monitoring

Most candidates stop at the architecture. Spend your last five minutes here and you are in the top quartile.



LAYER	METRIC	TARGET TO QUOTE
Retrieval	recall@10 on golden chunk ids, MRR	`recall@10 ≥ 0.9`
Answer	faithfulness, answer relevance (LLM judge, calibrated on 100 human labels)	`faithful ≥ 0.95`
Product	thumbs-up rate, deflection, follow-up rate, citation click-through	`trend, not absolute`
Ops	p50/p95 latency per hop, cost per query, error and refusal rate, cache hit rate	`p95 < 3 s · < $0.02/q`

Say "every answer stores its trace: query, rewritten query, chunk ids, scores, prompt, tokens, model version." That sentence answers five follow-ups at once.

The five-minute opening answer

Memorise the shape, not the words. This is what the first five minutes sound like when they go well.

0:00 **Restate.** "So: employees ask natural-language questions over about a million internal docs, answers must be grounded, budget-conscious. Correct?"

0:30 **Clarify.** "Five quick ones: how many users, how fresh, latency target, are there access controls, and can data leave our cloud?" Write answers down.

1:30 **Size.** "1M docs, ≈5M chunks, ≈30 GB indexed. 5K users, ≈3 QPS peak, ≈125M tokens a day. So a Postgres-scale index and a hosted API, no GPU fleet."

2:30 **Draw.** "Two paths. Offline: sources, parse, chunk with ACL metadata, embed, index. Online: gateway, hybrid retrieve with ACL filter, rerank, LLM with citations."

4:00 **Name the risks.** "The hard parts are parsing quality, permission leaks and hallucination. I would like to go deep on whichever you prefer."

4:45 **Hand over.** "Where do you want to dig in?" Let the interviewer choose. It shows confidence and buys you their agenda.

⏱ Practise it out loud with a timer, three times. The first run is always eight minutes.

Eight ways strong engineers fail this round

None of these are knowledge gaps. They are habits, and each one is fixable in a week of practice.

01 **Jumping to the vector database.** Tool names before requirements reads as junior. Ask first, choose later.

02 **No numbers at all.** "It scales" is not an answer. One estimate per component beats a perfect diagram.

03 **Fine-tuning as the first move.** For knowledge questions, retrieval wins on freshness, cost and citations. Fine-tune for style or format, later.

04 **Permissions as an afterthought.** If it comes up only when pushed, you have lost the security-minded interviewer.

05 **Agents everywhere.** A multi-agent crew for document Q&A is over-engineering. Earn complexity with a requirement.

06 **Never mentioning evals.** "How do you know it works?" should be answered before it is asked.

07 **Defending instead of adapting.** When the interviewer changes a constraint, they want to see the design move, not a debate.

08 **Running out of time on ingestion.** Parsing is fascinating. It is also 5 minutes, not 20. Watch the clock.

Six more prompts to rehearse

Same framework, different twist. For each, the twist is the thing the interviewer actually wants to hear you handle.

Customer support copilot

Twist: tool calls into order and CRM systems, human hand-off, PII in transcripts, a quality bar per intent.

AI code review bot

Twist: repo-scale context, diff-level retrieval, false-positive fatigue, a precision metric developers trust.

Voice agent for bookings

Twist: sub-second turn latency, STT/TTS or speech-to-speech, barge-in, confirmation before any write action.

Multi-agent research assistant

Twist: planner and workers, budget caps per task, durable state, when to stop, how to evaluate a report.

Recommendations with LLM rerank

Twist: candidate generation stays classical, LLM reranks the top 50, latency budget of ≈200 ms, offline A/B.

Invoice extraction pipeline

Twist: structured output with schema, confidence per field, human review queue, accuracy by field, not overall.

Rehearse one per day. Clarify, size, draw, pick two deep dives, close with evals. Record yourself once. It is humbling and it works.

The AI system design interview on one slide

STEP	REMEMBER
Format	45 min: clarify 5 · numbers 5 · design 10 · deep dives 15 · evals 5 · close 5.
Clarify	Users, data size and shape, freshness, latency SLO, permissions, budget, residency, what "good" means.
Numbers	1M docs ≈ 5M chunks ≈ 30 GB. 5K users ≈ 3 QPS peak ≈ 125M tokens/day ≈ $1–4K/month API.
Design	Offline: parse → 300–800 token chunks + ACL → embed (versioned) → index. Online: gateway → hybrid retrieve + ACL → rerank → LLM + citations.
Retrieval	Hybrid + RRF, pre-filter ACL inside the query, cross-encoder rerank 50 → 8.
Generation	Grounding rule, citations verified in code, confidence gate to refuse, fallback model, streaming.
Trade-offs	Recall vs precision, quality vs cost, freshness vs cache, latency vs rerank, API vs self-host. Say both ends.
Evals	Golden set in CI, recall@10, faithfulness judge, canary 5%, traces per answer, cost per query.
Avoid	Tools before requirements, no numbers, fine-tune first, agents everywhere, evals never mentioned.

Requirements, numbers, a diagram, two deep dives, trade-offs, evals. Same six moves, every AI system, every interview.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost